From Admitted Distinctions to S(n) and MSS-N: Presentations, Retractions, and Obstruction

Jeremy H. Carroll

2026-10-09

Version 1.0.0

Abstract

An admitted distinction raises two separate questions: how should it be represented, and which prior information must remain recoverable as the representation changes? This paper consolidates the S(n) presentation-tower account, the MSS-N mathematical monograph and the corrected Pre-Genesis-through-MSS-N architectural walkthrough. It distinguishes native admission stages, alphabet inventories, grammar profiles and atlas targets. It develops finite demand accounting, observation refinement, typed recovery on an old image, and conditional claims about minimal extensions. At the language level, alphabet inclusion admits an exact restriction retraction. At the chain-complex level, a stronger preservation promise has a sharp limit: a chain-map retraction makes the induced homology map injective, so an extension killing an old nonzero class cannot retain that retraction. Explicit graph examples show why syntactic recovery, observational discrimination and topological repair need different checks. These distinctions provide a bridge of questions and typed obligations into WDP; they do not claim that the complete WDP calculus or a physical theory has been forced from the initial inscription.

1. What "Before the First Mark" supplies

The primitive monograph *Before the First Mark* begins before interpretation of <>. It distinguishes absent input, the empty word, the literal inscription and an admitted ground. It explains why a task cannot distinguish two objects through a report that identifies them and why a locally valid reply can still depend on the wrong historical state.

Here those lessons become requirements on representations. A candidate extension must name what it changes, what it preserves and which problem it resolves. A representation's convenience, mathematical richness or physical analogy does not authorize its admission to an earlier regime.

The consolidation adopts two architectural rules:

1. A proposed native extension must be justified against an unresolved, declared obligation and the allowed response class.
2. A mathematical realization has no generative authority over that native regime.

These are policies for the programme. Whether a particular implementation enforces them requires evidence about that implementation. A theorem about one model does not establish estate-wide enforcement.

2. The indices and layers

Symbol or object	Meaning	Separate quantity
S(n)	A presentation stage in the specified admission account.	The number of glyphs or a homotopy degree.
k	Alphabet size under a declared token policy.	Number of decoded objects or operations.
p	Grammar/profile and public start.	Semantic interpretation or evaluation verdict.
MSS-m	An externally specified atlas target.	A native Genesis stage or a completed lift.
H_j	Homology in degree j of a declared complex.	Stage n or active-demand count.
C(m), D	A declared coverage measure and length of a contiguous licensed chain.	One another; neither is inferred from the number of folders.

For example, S(2) in the teaching crosswalk records retained but observationally collapsed data. MSS-2 is a target about relations. Equating them by their numerals silently changes the object under discussion.

Four responsibilities remain distinct: native admissions; mathematical representations; downstream interpretations; and verification/evidence machinery. A Lean proof, a Python calculation and a SQL view may refer to related structures, but shared terminology does not prove that they evaluate the same property.

3. The finite-stage crosswalk

The Genesis account is the source for admission, observation, candidate costs and the sealed/post-seal distinctions. The following is a presentation crosswalk, not a claim that the displayed glyphs derive their own semantics or that every row is forced independently of policy.

Stage	Presentation question	Required qualification
S(0)	What is received as the opaque ground inscription?	<> has no inherited object-language meaning. Ground inertia is relative to the specified rules.
S(1)	Has a newly supplied mark been consumed or admitted?	Supply alone may be an internal no-op.
S(2)	What data have been retained, and which profiles remain equal?	Host-level multiplicity does not create an internal distinguishing context.
S(3)	How can a supplied pair-separation task be met?	The cost-two result uses a particular response grammar and cost; it is not a universal witness count.
S(4)	Have the relevant pairs been separated under the chosen closure policy?	Pair closure needs its finite-carrier and preservation assumptions.

Stage	Presentation question	Required qualification
S(5)	Where does the next requirement or fresh datum come from?	External continuation is a declared policy, not an endogenous generator proved by saturation.
S(6)	How are old presentations represented and recovered after an extension?	Typed maps and their preservation laws must be supplied and checked.

A branching set of tied responses is not automatically a linear tower. A chosen cost order, equivalence and continuation policy select or retain its branches. Subsequent atlas targets may ask for graphs, algebra, categories or geometry. Their presence in the atlas does not establish that the native process has reached them.

4. Active demands, receipt sets and ordered histories

Let W_n be a finite register of active demands. A demand identifies its carrier, protected condition, witness data and activity predicate. It is not a bare label such as "curvature" or "missing structure." Let D_{n+1} be the demands discharged in a step and B_{n+1} the demands introduced. Assume

$$D_{n+1} \subseteq W_n, \qquad B_{n+1} \cap (W_n \setminus D_{n+1}) = \emptyset,$$

and define

$$W_{n+1} = (W_n \setminus D_{n+1}) \sqcup B_{n+1}.$$

Proposition 001.1 — finite demand accounting. With s_n=|W_n|, d_{n+1}=|D_{n+1}| and b_{n+1}=|B_{n+1}|,

$$s_{n+1} = s_n - d_{n+1} + b_{n+1}.$$

Proof. Subset removal gives |W_n minus D_{n+1}|=|W_n|−|D_{n+1}|. Disjoint union then adds |B_{n+1}|. □

This identity does not prove which demands an engine actually carries. In particular, the previously displayed sequence (0,1,1,1,0,1,1) follows from one chosen assignment of singleton demands to the crosswalk. It is an illustrative ledger, not a universal native survivor sequence or a physical observable.

A cumulative receipt set H_{n+1}=H_n union C_{n+1} has nondecreasing cardinality. An ordered event log additionally retains order and multiplicity. Neither is W_n. An empty active register need not erase receipts, imply thermodynamic equilibrium, or prove that every possible future question has been answered. Adding two map symbols does not imply two active demands or two newly admitted carrier elements.

5. Binary support and observation refinement

An inhabited irreflexive relation R(a,b) forces a≠b, as proved in Paper 000. This gives two endpoints for that edge, not two observers or two certificate occurrences. The pointed graph D2 with vertices 0 and 1 and the single edge 0→1 has exactly one endpoint-preserving map to a specified pair (a,b); that map preserves the edge. This elementary pointed property does not make D2 initial in an unspecified category of all possible systems.

Suppose q:A→O identifies a and b. If a supplied tag t:A→{0,1} has t(a)=0 and t(b)=1, define q'(x)=(q(x),t(x)). Then q'(a)≠q'(b), while projection to the first component recovers q. Both claims follow directly from equality of ordered pairs. The tag and its construction are supplied data. A method for finding tags is a separate algorithmic obligation.

For several required pairs, form a conflict graph on A. A tag assignment separating every edge is a proper coloring. Thus the number of tag values needed for many pairwise constraints need not remain two: a three-clique needs three colors. The constraints remain binary without implying a globally binary solution space.

6. Typed recovery on the old image

Let u:A→B be a proposed lift and let

$$\hat{r} : B \to A + \{\text{NON_DESCENT}\}$$

be a typed recovery map. The old-image law is

$$\hat{r}(u(a)) = \text{inl}(a) \quad (a \in A).$$

Proposition 001.2 — injectivity. This law implies that u is injective. If u(a)=u(a'), applying the recovery map gives inl(a)=inl(a'), hence a=a'. □

The law alone says nothing about off-image inputs. A checker intended to reject all of them needs the additional specification

$$b \notin \text{im}(u) \implies \hat{r}(b) = \text{inr}(\text{NON_DESCENT}).$$

It also needs a way to decide the image condition within its declared domain and budget. That is immediate for some finite presentations and not a general computability theorem.

Two further qualifications prevent stronger promises from being inferred:

- **Observation preservation needs a commuting condition.** For old observation q:A→O and new observation q':B→O, require q' composed with u=q, or provide a declared transport to the old report space. Recovering a term does not by itself prove that a new evaluator uses the old observations.
- **Repair needs its own proof.** An identity embedding has a perfect old-image law and can leave every original obstruction unchanged.

The pair is not generally an involution. Moreover, u composed with r-hat is not even a map B→B until handling of the refusal summand has been defined. No global preservation of all certificates follows from a successful term round trip alone.

The retained SnCarrier model does not prove this law for its entire input type. The compiled checks refusal_token_round_trip_changes_input and carrier_card_table_disagrees_with_higher in SnCarrierDischarge.lean import that module. `sectionLift stage (.nonDescent ())` uses the fallback empty-body lift, and retractionDownMap then accepts a ground presentation instead of recovering the original refusal token. `Carrier.higher 0 0` inhabits `Carrier 0`, while `carrierCard 0 = 0`. The table is not a cardinality theorem about that inductive type. These compiled facts discharge the inspection. They do not prove a stage-specific admissible domain or a corrected cardinality statement. Those repair obligations stay open, and the unrestricted old-image law is not claimed for this model.

7. Formal languages and conditional expressive capacity

A symbol system must declare terminal spellings, tokenization, grammar, public start, layout policy, identity policy, annotation decoding and allowed manipulations. These eight choices determine what an inscription does.

For the familiar alphabet Gamma_38, the glyph inventory comprises 26 lowercase Latin letters, ten decimal digits and <, >. Some MSS grammar profiles add punctuation or Unicode; they are not confined to these 38 characters.

Keep three relations separate:

$$E \subseteq V \times V, \qquad \to_{\text{rewrite}} \subseteq \Sigma^* \times \Sigma^*, \qquad q : V \to O.$$

They represent an encoded relation, rewrites between inscriptions and an observation map respectively. A change in any one need not change the others.

With repetition admitted, the one-letter language a+ supports positive unary counting under the length decoder. Including epsilon gives a* and a representation of zero. With two letters, concatenation is noncommutative: ab and ba are different words. A declared binary decoder can use any two distinct printed letters. A relation may be encoded by positions or a table without an arrow glyph. These are conditional constructions, not semantic resources supplied by alphabet size alone.

7.1 Free monoids and numeral decoders

Theorem 001.3. Free monoids on finite alphabets of different sizes are not isomorphic.

Proof. In a free monoid, the nonidentity elements that cannot be factored as a product of two nonidentity elements are exactly the one-letter words. A monoid isomorphism preserves identity and this indecomposability property in both directions. It therefore induces a bijection of the generating alphabets. Their sizes must agree. □

Thus binary and decimal string monoids are not isomorphic. Decoders can map their strings onto the same natural numbers without making the string monoids identical. A fixed-width binary encoding of decimal digits is injective under concatenation, but not onto the binary free monoid.

Likewise, discrete categories with two and 26 objects are not equivalent: all isomorphism classes are singletons, and an equivalence would have to match those classes bijectively.

7.2 Three recovery constructions with different types

For alphabets A contained in B, language inclusion and restriction are

$$\ell : \mathcal{P}(A^*) \to \mathcal{P}(B^*), \quad \ell(L) = L, \qquad r(M) = M \cap A^*.$$

Then r(ell(L))=L. This is a retraction of sets of words.

For individual words, exact recovery of old words can instead return the same word if it is in A, *and a typed refusal otherwise. A total erasure homomorphism h:B→*A* is another choice: map each new letter to epsilon and concatenate. It recovers every old word but loses new letters and their positions. These are three distinct contracts, not one overloaded "down-map."

No abelian category of ordinary formal languages has been specified here. An Ext calculation cannot be inferred from language restriction. Even in an abelian category, a split extension is one zero extension class; it does not show that an entire Ext group vanishes.

8. Manipulations and actual information loss

A renaming f:A→B has a two-sided inverse on the entire codomain B exactly when it is bijective. Recovery of source letters from the image f(A) only requires injectivity. Mapping both c and d to d collapses ac and ad. This is independent of whether the resulting word is grammatically well formed.

For the code a↦0 and b↦00, the different source words aa and b both encode as 00; ab and ba both encode as 000. A decoder cannot recover the source universally without additional retained information or a restricted source domain.

For declared directed-edge syntax:

Inscription	Represented edge
`a>b`	a→b
`b>a`	b→a
`a<b`	b→a
`b<a`	a→b

Reversing the vertex sequence and every arrow in `a>b>c` gives `c<b<a` and preserves its directed edge multiset. It changes the traversal order. A checker must name whether it protects the edge multiset, the ordered trace, annotations, or exact source bytes. AST recovery and byte recovery are different questions.

9. Finite graphs and the homological obstruction

For a finite directed graph, regard the underlying graph as a one-dimensional cell complex and define

$$C_1 = \mathbb{Z}^E \xrightarrow{\partial_1} C_0 = \mathbb{Z}^V, \qquad \partial_1(e) = t(e) - s(e).$$

If V is nonempty and the underlying undirected graph has c components, then H_0 is isomorphic to Z^c and reduced H_0 to Z^{c−1}. The maximal component count |V| occurs when no edge joins distinct vertices; self-loops do not affect it. Adding an edge reduces the component count only when it joins previously distinct components. It does not separate a and b under an unchanged observation q with q(a)=q(b).

Formal negative coefficients do not authorize reverse physical traversal. In a CW graph, a self-loop has zero boundary and can generate nonzero H_1. It is not a degenerate simplex solely because its endpoints coincide; the model matters.

9.1 The chain-retraction obstruction

Theorem 001.4. Let i:C→D and r:D→C be chain maps with r composed with i =id_C. Then H_j(i) is injective for every j. In particular, i cannot kill a nonzero homology class of C.

Proof. If z is a cycle in C and i(z)=partial_D b, then

$$z = r(i(z)) = r(\partial_D b) = \partial_C r(b).$$

So z was already a boundary in C. Therefore the kernel of H_j(i) is zero. □

This is the central preservation limit inherited from the MSS-N monograph. It does not prohibit every useful extension; it identifies incompatible promises. An extension may repair a cycle while preserving inscriptions, labels or selected questions, but it cannot simultaneously claim this chain-map retraction fixing all of C.

9.2 Two isolated points

Let C_0=Za plus Zb and C_1=0. Add an edge e with partial_D e=b−a, obtaining D. A retraction fixing the original vertices would have r_0=id and r_1(e)=0. The chain-map equation would require b−a=partial_C(0)=0, which cannot hold in the free abelian group on a and b. The new edge kills the old reduced zero-dimensional class, so the theorem applies.

9.3 A triangle and an explicitly attached face

Under label identity, `a>b>c>a` represents edges e_ab, e_bc and e_ca. Their sum z has boundary (b−a)+(c−b)+(a−c)=0. With no two-cells, H_1 is generated by [z]. Adjoining a face f with partial_2 f=z makes z a boundary and satisfies partial_1 partial_2=0. The inscription gave the one-skeleton; the face is an additional declared object.

Killing this class prevents a chain-map retraction fixing the original complex. It does not prove that an unrelated observational collision has been resolved. Nor does one vanishing class show that every homology group vanishes.

Chains and cochains must remain typed separately: C^j=Hom(C_j,Z) and (d^j phi)(c)=phi(partial_{j+1}c). Filling a chain is not the same equation as finding a cochain primitive. Homological vanishing also differs from null-homotopy: the commutator loop in the wedge of two circles has zero abelianized homology class but is nontrivial in its free fundamental group.

10. Profiles, parsers and evidence scope

The MSS grammar defines the 22 explicit starts inventoried in Appendix A. It supplies formation rules, not physical models or admitted semantic operations.

Selected profile	`a>b>c>a`	`a0>b2>c1>a0`	Interpretation if a decoder is supplied
raw_glyphs	Accepted	Accepted	Character sequence only.
label_path	Accepted	Accepted	Whole labels such as a0 are identifiers.
suffix_path	Rejected	Accepted	A letter plus a required digit suffix.
glyph_path	Accepted	Rejected	Single glyphs separated by direction marks.

An outer enclosure is not a universal requirement. Profile choice is part of the input contract; a parser must not guess a favorable interpretation after seeing an otherwise invalid candidate. Byte sequences, Unicode scalar values, grapheme clusters and grammar tokens also need separate counting conventions.

The historical inventory describes 22 starts alongside six retained runtime profiles. That inventory is not evidence of 28 semantic evaluators or a single shared resource limit. Recognition, AST construction, decoding, evaluation and exact serialization are distinct checks. A scalar syntax that accepts `a:1/0` still needs a decoder to reject zero denominators unless the grammar itself has been changed to exclude them.

The source archive preserves the original parser descriptions and historical codec example. They must be bound to their profile and implementation revision before being used as current protocol requirements.

11. What a justified lift must demonstrate

For a claim that an extension is necessary or minimal:

1. State the failed predicate and protected observations or laws.
2. Specify admissible repairs within the current carrier and operations.
3. Establish that those repairs cannot meet the predicate, or report incomplete search if the budget prevents that conclusion.
4. Specify the extension class, equivalence and cost order.
5. Retain all tied minima unless an explicit selection policy is applied.
6. Check that each accepted extension meets the predicate and preserves the particular contract being claimed.

A finite candidate census is exhaustive only for its declared domain. A budget stop is not proof of obstruction. A declared successor, a proposed lift and a licensed transition therefore have separate evidence states. Exact names of runtime terminal values belong to the engineering protocol's schema; combining papers does not create a new enum or claim every runner already conforms.

Similarly, proposer, evaluator and authorizer are roles with different inputs and responsibilities. Three agents are neither necessary nor sufficient. A theorem that an already halted state is unchanged by replay establishes that equation; it does not prove the authorization of an active transition.

12. Boundaries retained from the corrected walkthrough

The following withdrawn claims must not re-enter through consolidation:

Earlier inference	Correct scope
Two-sided inverse carrier maps establish univalence.	They establish the stated equivalence. A univalent universe requires a different construction and proof.
S(n) is automatically a Postnikov tower.	Construction stage and truncation degree differ; maps and preservation properties of a fixed-space Postnikov tower are extra data.
A selected CHSH expression detects all contextuality.	All required orientations and the model assumptions must be checked. A Bell score alone is not a constructed Cech cohomology class.
$\texttt{F}^2\texttt{=I}$ establishes a pentagon identity.	It is an involution check; full associator coherence is a different statement.
A SQL comparison and a Lean compilation establish bisimulation.	A relation connecting their states and transitions still needs a proof.
Small matrix/graph fixtures certify a physical theory.	They establish only the checked finite calculation, within their arithmetic model and input assumptions.

For the correlator tuple (1,1,−1,1), the selected expression E00+E01+E10−E11 is zero, while E00+E01−E10+E11 is four. Uniform-marginal pairwise distributions with those correlations exist. No global deterministic assignment can satisfy all four signs: the product of the four required pair products is −1, but every assignment gives +1. This is the concrete missed orientation from the corrected walkthrough.

The physics cleanup record records removed fabricated implementations and repaired finite examples. Valid trace, determinant, commutator, boundary and finite graph calculations remain useful. Their utility does not reinstate removed CDT, Super-WDW, sign-problem, chiral-superconductor or universal physical-verification claims. This paper does not claim a fresh rerun of those implementations.

13. The bridge into WDP

The common question across the preceding examples is now explicit. Let q:X→Y be a report and f:X→Z a protected answer. Can the report answer the question? The criterion is that f be constant on every fiber of q. Equivalently, there is a unique function g on im(q) with f=g composed with q.

The image restriction matters. Extending g to all of Y requires values on unrealized reports; it need not be unique and can fail if the codomain has no available value. Paper 002 develops report descent,

finite observation costs, linear obstruction, calibrated recovery and other conditional constructions with their own premises and evidence.

Object here	Question entering Paper 002
A projection of history	Is the protected state constant on its fibers?
A glyph/profile decoder	Which distinctions survive decoding or normalization?
An old-image recovery map	Which source inputs are recoverable, and where must it abstain?
A cochain or quotient representation	Does the protected question descend, and what witnesses failure?
A candidate repair	Does it meet the condition while preserving the declared observations?

This is an explanatory bridge with explicit mathematical interfaces. A complete native derivation of WDP from an opaque inscription remains a separate claim; this trilogy does not supply it by changing its publication order.

14. Evidence and consolidation record

The claim ledger records the scope of each main result. The source map assigns the three predecessor papers to the sections above and names claims that were narrowed or omitted. The source archive preserves those manuscripts and their ledgers/change logs. The education paths now point here; they are not competing active manuscripts.

The source manifest records the baseline, original blob identities and digests, and relocated archive digests. Source links were adjusted for the archive location. Local Lean 4 proofs live in lean4, and the finite Python checks live in code/sn_mssn_checks.py. The reproduction entry is code/reproduce.py. This revision makes no new model-call, full-repository coverage, DOI, or publication-deposit claim.

Appendix A. The 22 declared MSS grammar starts

This table is the source inventory, not a list of completed semantic adapters. The production grammar remains the authority for accepted syntax.

#	Start Symbol	EBNF Production	Canonical Input	Lexical & Structural Contract
01	raw_glyphs	`{ glyph | "<" | ">" }`	`a><b0<<`	Raw strings over Γ_{38}, including ε. No balancing required.
02	word	`glyph, { glyph }`	alpha01	Nonempty sequence over Σ_{36}. No delimiters.

#	Start Symbol	EBNF Production	Canonical Input	Lexical & Structural Contract
03	glyph_path	`glyph, { direction, glyph }`	`a>b<c>a`	Alternating single glyphs and directions.
04	label_path	`word, { direction, word }`	`a0>b2>c1`	Directed paths between multi-glyph identifiers.
05	suffix_path	`suffix_vertex, { direction, suffix_vertex }`	`a0>b12<c3`	Vertex structured as lower_latin followed by digits.
06	path_list	`glyph_path, { ";", glyph_path }`	`a>b;c<d`	Semicolon-delimited sequence of independent paths.
07	suffix_list	`suffix_path, { ";", suffix_path }`	`a0>b1;c2<d3`	Semicolon-delimited sequence of suffix paths.
08	scalar_path	`scalar_vertex, { direction, scalar_vertex }`	`a:1.5>b:-2/3`	Paths with colon-separated scalar weights. `a:1/0` parses syntactically.
09	vector_path	`vector_vertex, { direction, vector_vertex }`	`a[1,2]>b[3,4]`	Bracketed vector-annotated vertices.
10	edge_weight_path	`lower_latin, { weighted_link, lower_latin }`	`a>2.5>b<1<c`	Arrow-flanked edge weights (`>w>` or `<w<`). Opposing marks (`>w<`) rejected.
11	labelled_path	`lower_latin, { "-", word, ">", lower_latin }`	`a-step>b`	Named relation links (`-label>`).
12	prefix_tree	`glyph | direction, prefix_tree, prefix_tree`	`>ab`	Polish prefix notation for binary syntax trees.

#	Start Symbol	EBNF Production	Canonical Input	Lexical & Structural Contract
13	postfix_tree	`glyph, { postfix_tree, direction }`	`ab>`	Reverse Polish notation for binary syntax trees.
14	sequence	`glyph, { ",", glyph }`	`a,b,c`	Comma-delimited list of single glyphs.
15	nested_tree	`glyph | "(", [ nested_tree, { ",", nested_tree } ], ")"`	`(a,(b,c))`	Parenthesized nested trees.
16	head_term	`word, [ "(", [ head_term, { ",", head_term } ], ")" ]`	`f(x,g(y))`	Functional terms with head words and argument lists.
17	incidence	`incidence_item, { ";", incidence_item }`	`[a,b]>[c,d]`	Hypergraph directed incidence lists.
18	unicode_path	`unicode_glyph, { unicode_direction, unicode_glyph }`	→←	Directed paths over Greek/symbolic Unicode alphabets.
19	arrow_path	`glyph, { unicode_direction, glyph }`	a→b←c	ASCII glyphs joined by Unicode arrows (→, ←).
20	bitstream	`bit, { bit }`	10011010	Nonempty binary digit strings (0 and 1).
21	octets	`hex_digit, hex_digit, { hex_digit, hex_digit }`	deadbeef	Lowercase hexadecimal byte strings (even length required).
22	rewrite_rule	`{ glyph }, "=>", { glyph }`	`a=>b`	String rewrite rules with optional empty sides.